

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are rapidly being integrated with graph-structured data for tasks like node classification, a domain long dominated by Graph Neural Networks (GNNs). This integration exploits rich relational information to boost task performance, yet their robustness against adversarial attacks has remained entirely unexamined. We deliver the first systematic study of the vulnerabilities of graph-aware LLMs, subjecting two representative models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024), to adversarial attack methods spanning both poisoning (training-time attacks) and evasion (test-time attacks). Beyond transferring existing attacks, we uncover a new attack surface unique to LLAGA, in which an attacker injects malicious nodes as placeholders into the node sequence template to sharply degrade its performance. Our analysis establishes that specific design choices in graph encoding directly amplify attack success, and we show conclusively that: (1) the node sequence template in LLAGA is a decisive source of vulnerability; (2) the GNN encoder in GRAPH PROMPTER confers markedly greater robustness; and (3) both approaches are acutely susceptible to imperceptible feature perturbation attacks. Finally, we introduce GALGUARD, an end-to-end defense framework that couples an LLM-based feature correction module for feature-level perturbations with adapted GNN defenses for structural attacks, and demonstrate that it restores robustness across all settings¹

1 Introduction

Graphs, characterized by nodes and edges that represent entities and relationships, are used to model complex structures in various real-world domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and health-care (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), designed specifically for graph-structured data, excel in tasks such as node classification and link prediction. Recently, the success of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023), trained on vast amounts of textual

data, has sparked interest in their potential to enhance graph-related tasks. This fusion of graph data with LLMs (called *graph-aware LLMs* (Xie et al. 2023)) has opened a rapidly expanding line of research, in which LLMs increasingly take over tasks traditionally handled by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Indeed, recent work demonstrates that graph-aware LLMs outperform GNNs on a range of graph-related tasks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), establishing them as compelling alternatives.

Despite this momentum, the vulnerabilities of graph-aware LLMs to attacks have gone entirely unexamined. Existing adversarial attacks on GNNs, spanning poisoning (training-time attacks) and evasion (test-time attacks) (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), operate in a black-box setting where the attacker has no knowledge of the model’s internal workings. These attacks undermine *model integrity*, deliberately degrading performance by subtly altering graph structure or node features. GNN research has fixated almost exclusively on structural attacks; yet the textual nature of graph-aware LLMs makes imperceptible perturbations to node features a serious, and thus far overlooked, threat.

Three observations drive this work: the decisive shift from GNNs to LLMs for graph tasks, the black-box nature of current adversarial attacks on GNNs, and the new vulnerabilities that feature perturbations introduce. Together they compel a critical question: *do adversarial attacks designed for GNNs remain effective against the emerging graph-aware LLM paradigm?* Answering it matters because the success of graph-aware LLMs on graph tasks *hinges* on how effectively graphs are integrated into the LLM. Processing node features is straightforward given the LLM’s proficiency with text, but incorporating graph structure is far more demanding and requires purpose-built techniques to capture relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). To answer the question rigorously, we first categorize existing graph-aware methods into two classes by how they encode graphs into LLMs: one representing graph data as *textual descriptions* that convert graph information into natural language, and another employing *learned projectors* to bridge the inherent gap between graph and text

¹Our code will be released upon acceptance.

domains. Since conventional text-based adversarial attacks already suffice to compromise LLMs that consume textual descriptions of graphs, we concentrate our investigation on the learned projector methods, using two computationally efficient representative models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024), both built on a frozen pre-trained LLM.

Our analysis establishes that specific design choices in graph encoding actively amplify the success of attacks. Most strikingly, we identify a new attack surface in LLAGA that lets an attacker severely degrade the model’s performance by injecting malicious nodes as placeholders within the node sequence template. Building on these findings, we propose an end-to-end graph-aware LLM defense, GALGUARD, which integrates an LLM-based feature corrector with adapted GNN-based structural defenses to deliver robust protection. To the best of our knowledge, this is the first investigation of the adversarial vulnerability of graph-aware LLMs. The contributions of the paper are summarized as follows:

- We leverage representative adversarial attacks for GNNs, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to establish the transferability of poisoning and evasion attacks to graph-aware LLMs.
- We introduce a taxonomy for graph-aware LLMs based on their encoding approaches, clarifying precisely how different methodologies integrate graph structures into LLMs.
- We discover a new attack surface in LLAGA, demonstrating that injecting malicious nodes as placeholders into the node sequence template sharply degrades its performance.
- We deliver the first holistic assessment of graph-aware LLM vulnerabilities by mounting imperceptible attacks on textual node features, a perspective GNN research has overlooked.
- We propose GALGUARD, a novel end-to-end defense strategy that unifies LLM-based feature correction with the adaptation of existing GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs fall into two main approaches: *textual descriptions of graphs* (where graph information is encoded as natural language) and *learned graph projectors* (where a projector encodes graph data into embeddings or structured forms that LLMs can directly process). We describe the encoding approaches in each category below. Figure 5 illustrates both approaches. Full description in Appendix A.1 and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. This approach converts graph data into natural language, enabling LLMs to process them as textual descriptions, often with specific instructions to query LLMs (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In multimodal applications, Liu et al. (2023) and Su

et al. (2022) linked molecule graphs with textual descriptions. Furthermore, Huang et al. (2023a) investigated incorporating graph structure into LLM prompts. Their findings reveal that LLMs process graph-related prompts as contextual paragraphs, underscoring the decisive importance of prompt design, as seen in studies like (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other research uses LLMs to enhance node features and graph structures, act as predictors, or serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs, and Duan et al. (2023) introduced an effective approach that enhances textual graph learning. While these methods offer a straightforward route to using LLMs for graph tasks, they fail to fully capture structural intricacies (especially for large graphs) and depend heavily on prompt engineering.

Graph Representation as a Learned Projector. To address the limitations of textual representations, learned projector methods transform graph data into embeddings or graph vectors that LLMs can process more directly. This includes methods like LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), and others that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other approaches use graph neural prompts to encode knowledge graphs (Tian et al. 2024), develop prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or condense graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We focus on projector methods that utilize *frozen pre-trained LLMs* precisely because they are computationally efficient and sidestep the substantial costs of retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). For our investigation, we use two representative models: LLAGA (Chen et al. 2024), which employs node-level templates with a linear projector, and GRAPH PROMPTER (Liu et al. 2024), which uses a GNN with a linear projector. We deliberately exclude textual description methods: they scale poorly and, crucially, expose none of the novel, graph-specific vulnerabilities we set out to investigate.

3 Threat Model

We ground our threat model in established research on adversarial attacks in graphs and extend it directly to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The model reflects realistic deployments: in social networks, an attacker typically controls only a handful of bot accounts, exactly the limited access our threat model assumes, yet can still manipulate a small subset of nodes to evade detection or sway the classification of legitimate users. On Wikipedia, hoax articles with sparse connections to legitimate ones can be strategically linked to manipulate the classification of genuine content, forcing misclassification of real articles’ categories. This transferability is especially alarming because

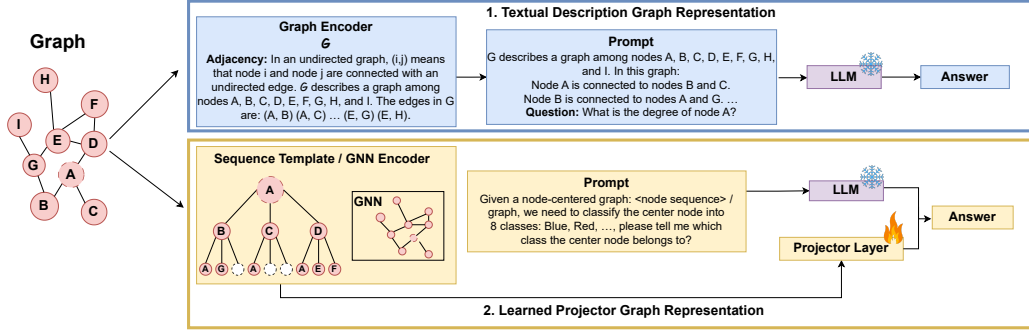


Figure 1: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

these advanced models are poised for deployment in critical applications such as recommendation systems, social media analysis, and even healthcare, where they may well replace GNNs. Following prior work on adversarial attacks in graphs, we characterize the threat model along three dimensions—model access, adversarial capabilities, and attack strategies—and then define the attacker’s goal.

Model Access. The attacker operates in a black-box setting, with no knowledge of the model’s architecture or parameters. The attacker can only influence the model indirectly by interacting with the training or test data.

Adversarial Capabilities. Consistent with existing poisoning (evasion) attacks, the attacker has complete access to the training (inference) data of the target model. Within a predefined budget and under unnoticeability constraints (aiming to avoid detection), the attacker can modify or inject data.

Attack Strategies. The attacker leverages two primary strategies: feature manipulation (altering node features) and structure manipulation (adding or removing edges).

Attacker’s Goal. The attacker’s goal is to compromise model integrity by degrading the overall performance, specifically by increasing the misclassification rate of the classification model.

4 Experimental Setup

Dataset. We evaluate on three widely used graph datasets: Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016). These text-attributed graph datasets, spanning small to medium scales and varying sparsity, are the standard benchmarks for models like those in our study (Chen et al. 2023a, 2024; Liu et al. 2024). To confirm scalability, we further include the large-scale ArXiv dataset (Hu et al. 2020) in an additional experiment (see Appendix A.5). We adopt the standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv, following (Chen et al. 2024). Detailed statistics are provided in Table 1.

Tasks. We adopt *node classification* as our primary task, the standard benchmark for evaluating graph machine learning models and the canonical setting for adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner

Table 1: Dataset statistics.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels, as our evaluation metric. Each experiment is repeated three times, and we report the mean and standard deviation. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now justify our selection of graph-aware LLM methods and adversarial attack methods for investigating the vulnerabilities of graph-aware LLMs.

Graph-Aware LLM Methods. We employ two representative graph-aware LLM methods: LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). Both build on a *frozen pre-trained LLM*, which makes them computationally efficient and readily adaptable to diverse graph tasks and LLM architectures. LLAGA uses node-level templates with a linear projector, while GRAPH PROMPTER employs a GNN-based encoder with a linear projector. In both cases, the projector is a 2-layer multi-layer perceptron (MLP). The foundational LLMs for LLAGA and GRAPH PROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We conduct our investigation with the poisoning and evasion attacks from two representative adversarial attack methods, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted attack that finds minimal perturbations to misclassify specific nodes, while METAATTACK mounts untargeted attacks by treating the graph structure as a hyperparameter to optimize. For NET-

Table 2: Performance of NETTACK and METAATTACK on the Cora, Citeseer, and PubMed datasets for the graph-aware LLMs LLAGA and GRAPH PROMPTER. Evasion attacks inflict far larger accuracy drops than poisoning across every dataset.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER	LLAGA	GRAPH PROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

TACK, we follow (Li et al. 2023; Zügner and Günnemann 2019), sequentially targeting each node and aggregating the results to yield an untargeted evaluation. In every experiment we perturb only 10% of the edges to honor the unnoticeability constraint, as in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Full details in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We analyze poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019). For poisoning attacks on LLAGA, we use a perturbed graph to generate the node-level sequences and then retrain the projector. For GRAPH PROMPTER, we use the perturbed graph to re-encode the graph with a GAT and retrain its projector. For evasion attacks, both models are trained on unperturbed data, and perturbations are introduced only during inference.

Results. Our experiments yield three clear conclusions, summarized in Table 2. First, poisoning attacks have limited impact on graph-aware LLMs, degrading performance by at most 11% under either attack method; the models’ robust representations and the additional context supplied by the LLM confer this resilience. Evasion attacks, by contrast, are devastatingly effective, driving performance down by as much as 51%. Second, METAATTACK consistently outperforms NETTACK for both poisoning and evasion across all datasets, a direct consequence of METAATTACK’s ability to craft small but highly effective perturbations by treating graph edges as hyperparameters. Third, denser datasets are decidedly more vulnerable than sparse ones: LLAGA suffers a 51% accuracy drop on the denser Cora under evasion, but only a 19% drop on the sparser PubMed.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Our experiments (Table 2) establish that LLAGA is decisively more susceptible to attacks than GRAPH PROMPTER, and pinpoint the cause: its *sensitivity to structural perturbations*. LLAGA integrates the graph structure in two ways—by generating a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each posi-

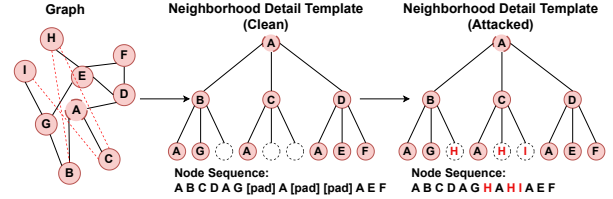


Figure 2: Illustration of the node sequence template injection attacks. The attack shown is the Non-adjacent Injection attack (NI); red dashed lines mark the attacker’s edge perturbations. For the Supernode Injection (SI) and Multiple Supernode Injection (MSI) attacks, the supernode(s) replace the placeholders directly. Here, $k = 3$.

tion in this sequence corresponds uniquely to a relative structural position within the original graph. Because the node embeddings are derived directly from the perturbed graph structure, these attacks corrupt the information processed by the LLM and degrade performance. GRAPH PROMPTER’s textual node embeddings, in contrast, stay intact under structural perturbations, since only the GNN component is affected. This design lets the unperturbed textual features reinforce the perturbed graph structure, which is precisely what makes GRAPH PROMPTER more robust.

6 Node Sequence Template Injection Attacks

We expose a new attack surface in LLAGA by exploiting the neighborhood detail template used to construct the node sequences. The attack is illustrated in Figure 2 and the algorithm is in Appendix A.4. To obtain the node sequence for a node u , LLAGA’s neighborhood detail template first constructs a *fixed-shape* computational tree for each node, where the children of a node are its sampled 1-hop neighbors. This process repeats to sample neighbors at each subsequent level of the tree. When a node has too few neighbors, a placeholder fills the remaining slots to preserve the fixed size of, e.g., $k = 10$ children, consistent with LLAGA’s original design (see “Clean” in Figure 2 and lines 1-14 in the algorithm). This fixed structure is exactly what opens a new attack surface.

Exploiting this neighborhood detail template algorithm, an attacker can inject malicious nodes into placeholder positions through graph manipulation and thereby sharply degrade model performance. We propose three attack strategies. The

Table 3: Results of the node sequence template injection attack across datasets and methods. *NI*, *SI*, and *MSI* denote the non-adjacent, single supernode, and multiple supernode injection attacks, respectively. Clean refers to model performance without any attack. Evasion attacks sharply degrade LLAGA on the smaller Cora and Citeseer graphs.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 $\pm$ 0.07	————
	Poisoning	0.86 $\pm$ 0.03	0.84 $\pm$ 0.06	0.82 $\pm$ 0.04
	Evasion	0.42 $\pm$ 0.08	0.36 $\pm$ 0.05	0.30 $\pm$ 0.05
Citeseer	Clean	————	0.71 $\pm$ 0.04	————
	Poisoning	0.69 $\pm$ 0.05	0.66 $\pm$ 0.03	0.62 $\pm$ 0.02
	Evasion	0.36 $\pm$ 0.06	0.30 $\pm$ 0.04	0.24 $\pm$ 0.05
PubMed	Clean	————	0.90 $\pm$ 0.04	————
	Poisoning	0.88 $\pm$ 0.04	0.85 $\pm$ 0.05	0.81 $\pm$ 0.03
	Evasion	0.85 $\pm$ 0.07	0.82 $\pm$ 0.04	0.76 $\pm$ 0.03

first, *Non-Adjacent Injection (NI)*, injects nodes lying outside the 2-hop neighborhood of a target node u (see “Attacked” in Figure 2 and lines 21-27). The second, *Supernode Injection (SI)*, injects a single high-degree node as a placeholder, again outside the target node’s 2-hop neighborhood. The third, *Multiple Supernode Injection (MSI)*, extends *SI* by injecting multiple distinct supernodes as placeholders (lines 35-43). The rationale is direct: inserting uninformative nodes into the node sequence corrupts the target node’s representation, yielding a poorly trained projection layer and compromising the integrity of both the model’s learning and its inference.

Results. As Table 3 shows, LLAGA withstands poisoning attacks on Cora and Citeseer, yet collapses under evasion: accuracy plunges to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the *NI*, *SI*, and *MSI* attacks, respectively.

On the larger PubMed dataset the model holds up far better against all injection attacks, giving up only a modest amount of accuracy under evasion (0.85, 0.82, and 0.76). The explanation is clear: in smaller, tightly connected graphs like Cora and Citeseer, a localized injection lands a concentrated blow that severely disrupts the model’s reasoning about the local neighborhood, whereas on the much larger PubMed graph the same attack is diluted and does far less damage.

Node injection attacks decisively outperform traditional graph adversarial methods. The *MSI* method drives evasion accuracy down to 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, respectively, while METAATTACK manages only 0.44, 0.50, and 0.73 (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Existing graph adversarial attacks concentrate almost entirely on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019), the sole exception being NETTACK (Zügner, Akbarnejad, and Günnemann 2018), which does not transfer to the textual domain. Because structural attacks are only moderately successful against robust models like GRAPHPROMPTER, and

because textual features are demonstrably central to model robustness, we target unnoticeable feature perturbation attacks to degrade model performance.

We adopt the imperceptible attack strategies of (Boucher et al. 2022) with one key modification: we drive the perturbations with a task-agnostic translation objective (English to French). The objective modifies the input text so that the changes are visually identical yet produce an incorrect translation, thereby corrupting the encoded semantics—if the translation of a text is wrong, its encoded semantics must be wrong as well. We employ two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar ones, and *Reorderings*, which use direction control characters to alter the rendering sequence. Examples of Homoglyphs and Reorderings are in Appendix A.6.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model’s output on the perturbed and unperturbed text. The perturbation budget is fixed at 10% of the average length of each textual feature.

Results. As Table 4 shows, Reordering attacks are consistently more potent than Homoglyph attacks across all models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both: poisoning drops accuracy by 27% and 23% under Homoglyph and by a still steeper 35% and 25% under Reordering, respectively. In every case, GRAPHPROMPTER proves the more resilient model. PubMed follows the same pattern, though the impact is smaller than on the smaller datasets.

Comparing against structural perturbations exposes a fundamental shift in vulnerabilities (Figure 3): feature perturbation attacks (specifically Reordering) outstrip structural attacks (METAATTACK) by a wide margin on both LLAGA and GRAPHPROMPTER. This establishes that graph-aware LLMs weigh input features far more heavily than graph structure, which is exactly what makes them so vulnerable to feature perturbations. We push this insight further by combining both attack types in a unified setting in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that fuses feature and structural perturbations into a single, more imperceptible and more effective adversarial strategy. Our hypothesis is direct: combining these attack types degrades model accuracy far more than either method alone, all while keeping the alterations to graph structure and features imperceptible.

For the structural perturbation we use METAATTACK, which subtly rewires edges. For the feature component we combine the Homoglyph and Reordering attacks from the previous section. Together, these perturbations form a highly imperceptible attack that strikes simultaneously at the relational and textual information on which graph-aware LLMs rely.

Results. As Table 5 shows, the unified attack—jointly applying structural and feature perturbations—is far more effective than either perturbation alone at degrading model performance across every dataset. This result delivers a key insight

Table 4: Performance of feature perturbation attacks, with percentage drop from clean accuracy in parentheses. Reordering attacks are the most damaging across all models and datasets.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 ± 0.05 (27%)	0.52 ± 0.04 (13%)	0.41 ± 0.04 (54%)	0.38 ± 0.03 (37%)
	Reordering	0.58 ± 0.06 (35%)	0.45 ± 0.05 (25%)	0.24 ± 0.05 (73%)	0.20 ± 0.03 (67%)
Citeseer	Homoglyph	0.54 ± 0.04 (23%)	0.65 ± 0.04 (7%)	0.32 ± 0.03 (55%)	0.52 ± 0.05 (26%)
	Reordering	0.46 ± 0.03 (35%)	0.39 ± 0.02 (44%)	0.20 ± 0.05 (72%)	0.27 ± 0.02 (61%)
PubMed	Homoglyph	0.72 ± 0.04 (20%)	0.83 ± 0.04 (8%)	0.36 ± 0.06 (60%)	0.55 ± 0.03 (39%)
	Reordering	0.67 ± 0.05 (26%)	0.80 ± 0.03 (11%)	0.28 ± 0.04 (69%)	0.42 ± 0.04 (53%)

Table 5: Model accuracy under the unified imperceptible attack, with percentage drop from clean accuracy in parentheses. The unified attack drives both models to near-collapse.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 ± 0.05 (61%)	0.28 ± 0.08 (53%)
	Evasion	0.14 ± 0.06 (84%)	0.11 ± 0.05 (82%)
Citeseer	Poisoning	0.26 ± 0.04 (63%)	0.22 ± 0.03 (69%)
	Evasion	0.10 ± 0.09 (86%)	0.13 ± 0.07 (81%)
PubMed	Poisoning	0.39 ± 0.07 (57%)	0.32 ± 0.04 (64%)
	Evasion	0.12 ± 0.05 (87%)	0.19 ± 0.04 (79%)

into the architecture of graph-aware LLMs: unlike traditional GNNs, which are chiefly affected by structural perturbations, these models are acutely vulnerable to feature-level attacks. The effectiveness of the unified attack confirms that graph-aware LLMs depend on the consistency of both graph structure and node features, leaving them highly susceptible to combined, inconspicuous modifications.

9 Ablation Study

9.1 Varying Feature Perturbation

To pin down the relationship between perturbation level and attack success, we vary the percentage of perturbed features. Our analysis of the Reordering attack, which mirrors the Homoglyph attack, reveals a consistent trend as the perturbation percentage moves from 1% to 10%. Across all models and datasets (Cora, Citeseer, and PubMed), reducing the fraction of perturbed features weakens the attack, as expected. Notably, as the perturbation level falls, the success rates of poisoning and evasion attacks converge—because with fewer perturbations the adversarial signal is simply too weak to move the model, whether it was injected during training (poisoning) or inference (evasion). A detailed plot of these trends appears in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We test whether the foundation LLM governs adversarial robustness in graph-aware LLMs. LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPHPROMPTER uses Llama2-7B (Touvron et al. 2023). We swap LLAGA’s backbone for Llama2-7B and compare performance under poisoning and evasion attacks on all datasets. Both models reach essentially the same clean accuracy (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora) and show equivalent vulnerability under attack, and the same holds when swapping LLMs in GRAPHPROMPTER. We conclude that the choice of foundation LLM has negligible impact on adversarial robustness. The full results are in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses against adversarial attacks are fundamentally inadequate for graph-aware LLMs, because these

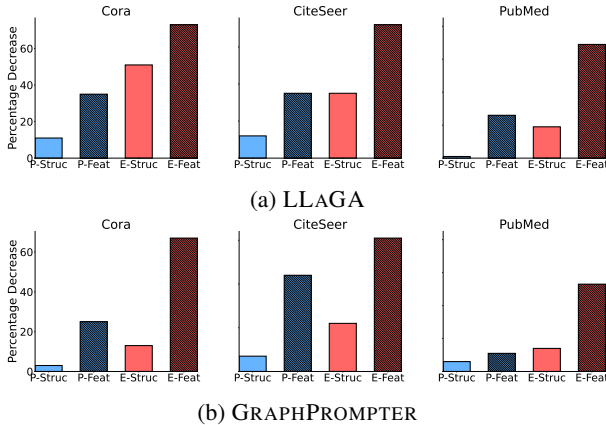


Figure 3: Performance of structure- and feature-based adversarial attacks on graph-aware LLMs. Attack success is measured by the percentage decrease (y-axis) from the original accuracy. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural and feature perturbation. Feature-based attacks dominate structural ones for both models.

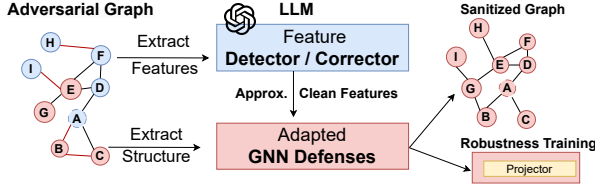


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are colored blue and red, respectively. The LLM detects perturbations in node features extracted from an adversarial graph and generates a corrected version. The refined features, together with the perturbed edges, are then passed through existing GNN defenses, which either yield a sanitized graph or are adapted for the projectors’ robustness training.

models are vulnerable to structural *and* feature perturbations at once. Many defenses lean on node feature similarity to prune or reweight edges (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), but they presuppose clean features; once features are perturbed, these methods fail outright (see Figure 3 in the Appendix). Defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), and even OCR-based sanitization (Boucher et al. 2022), likewise collapse under coordinated attacks. This dual vulnerability demands a new, integrated defense.

We answer that demand with graph-aware LLM defense (GALGUARD), an end-to-end defense strategy that unites two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, an LLM acts as an adaptive mechanism that scans textual node features for anomalous patterns and issues approximate corrections to restore feature integrity, neutralizing feature-level perturbations. This LLM-based correction is a practical, effective solution to feature vulnerabilities. The specific instructions and prompts are in Appendix A.6. For our experiments, we employ GPT-4 Turbo as the LLM corrector.

Graph Structure Robustness Strategy. To harden structural robustness, we use a two-step approach. First, we apply a *graph purification heuristic* (Wu et al. 2019) as a preprocessing step: for each node v , we compute the cosine similarity between its embedding $\phi(v)$ and its neighbors’ embeddings $\phi(x_u)$ and remove low-similarity edges. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020), applying its principles directly to the message-passing layers of GRAPH PROMPTER. For LLAGA, we introduce a novel adaptation: we fold the purification step into the construction of the computational tree for its node sequence, and we add a learnable *global structural context embedding* (M_{global}) as a parameter of LLAGA’s projector. By concatenating M_{global} with the purified node sequence embedding and training them jointly, the model learns to operate robustly on perturbed graph inputs. The plug-and-play design of GALGUARD lets other defenses, e.g., Pro-GNN (Jin et al. 2020), drop in place of GNNGuard.

10.1 Results

Table 6 reports accuracy under METAATTACK alongside our two defense variants: GALGUARD_p, which pairs the feature corrector with a graph purification module, and the full GALGUARD, which combines the feature corrector, the graph purification module, and GNNGuard (or its adaptation). Across every evaluated dataset (Cora, CiteSeer, and PubMed) and for both poisoning and evasion attacks, our defenses consistently blunt the impact of METAATTACK. GALGUARD_p already delivers a strong foundation, showing that correcting feature perturbations with the LLM corrector and applying initial graph purification substantially improve robustness. The full GALGUARD goes further still, consistently surpassing GALGUARD_p and confirming the synergistic gain from integrating the GNN-inspired structural defense.

Table 6: Performance of METAATTACK and the proposed defense GALGUARD under poisoning and evasion attacks. GALGUARD_p combines feature correction with graph purification. The full GALGUARD (bold) yields the strongest robustness across all datasets and attack types.

Data	Method	LLAGA		GRAPH PROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79±0.03	0.44±0.06	0.58±0.05	0.52±0.04
	GALGUARD _p	0.82±0.05	0.62±0.04	0.59±0.03	0.56±0.04
	GALGUARD	0.85±0.04	0.83±0.04	0.60±0.05	0.59±0.03
CiteSeer	MetaAttack	0.63±0.04	0.50±0.03	0.65±0.06	0.56±0.03
	GALGUARD _p	0.64±0.03	0.56±0.02	0.67±0.05	0.60±0.06
	GALGUARD	0.67±0.04	0.62±0.04	0.68±0.04	0.64±0.05
PubMed	MetaAttack	0.89±0.08	0.73±0.04	0.85±0.02	0.77±0.03
	GALGUARD _p	0.90±0.06	0.78±0.03	0.85±0.04	0.81±0.04
	GALGUARD	0.90±0.01	0.87±0.02	0.89±0.02	0.89±0.01

11 Conclusion

In this paper, we deliver the first systematic account of the adversarial vulnerabilities of graph-aware LLMs, a previously unexplored area. Our work establishes that the methods used to integrate graph data are themselves a critical source of vulnerability. We uncover new attack surfaces and show that design choices like the node sequence template in LLAGA sharply increase a model’s susceptibility. We further demonstrate that feature perturbation attacks are strikingly effective—unlike on traditional GNNs—routinely outperforming structural attacks, and that a unified attack combining both perturbations renders graph-aware LLMs all but useless.

To counter these threats, we propose GALGUARD, an end-to-end defense that unites an LLM-based feature corrector with adapted GNN-based structural defenses and restores robust protection across all settings. As these models proliferate across applications spanning social networks, healthcare, and finance, ensuring their resilience to adversarial exploitation is imperative. We are confident this work will serve as a foundation for further research, driving deeper investigation into the vulnerabilities and defenses of graph-aware LLMs

and yielding both theoretical insight and practical safeguards for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.

- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejdil, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.

- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is organized as follows. Appendix A.1 provides a detailed taxonomy of graph-aware LLMs. We discuss related work on attacks against Graph Neural Networks (GNNs) in Appendix A.2. Appendix A.3 gives an overview of the adversarial attacks, NETTACK and METAATTACK, while Appendix A.4 details the algorithm for the node sequence injection attack on LLAGA. We present the results of our ablation studies in Appendix A.5, including the effect of varying the amount of perturbed features and the choice of base LLM on attack performance. Finally, Appendix A.7 shows examples of feature attacks and the template for the LLM corrector, and Appendix A.8 discusses the implications of our research.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data is a fast-growing area of research, with a wide range of approaches adapting or enhancing LLMs for graph-related tasks such as reasoning, representation, and inference. These approaches divide cleanly into two categories, defined by how they encode graphs for LLMs: *textual descriptions of graphs*, where graph information is encoded as natural language, and *learned graph projector*, where a projector encodes graph data into embeddings or structured forms that LLMs can directly process. This section details the methods within each encoding approach. Figure 5 illustrates both approaches, and Table 7 summarizes their differences.

Graph Representation as a Textual Description In this first category, graph data is represented as natural language, allowing LLMs to process graph structures as textual descriptions. This approach has been explored extensively with works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) converting graph data into textual formats, often paired with specific instructions to query LLMs. For example, Tan et al. (2023) introduced a unified framework for attributed graph embedding by generating random walk-based textual sequences from graphs, which are then used to fine-tune LLMs. Similarly, Zhao et al. (2023) focused on graph reasoning within text spaces, translating graph structures into natural language to facilitate LLM interaction. Extending this idea, Wei et al. (2024a) augmented textual representations of graphs with visual representations of the graph, enhancing the reasoning capabilities of LLMs on graph data. In multimodal applications, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions, enabling advances in drug discovery and molecular retrieval. Furthermore, Huang et al. (2023a) investigated how incorporating graph structure into LLM prompts can enhance predictive performance. Their findings revealed that LLMs tend to process graph-related prompts as contextual paragraphs, rather than explicitly recognizing graph structures. This further underscores the importance of prompt design, as seen in studies like (Brannon et al. 2023; Zhang et al.

2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024), which extended LLMs to heterogeneous and large-scale graph tasks. Complementing these efforts, (He et al. 2024) explored using LLM-generated explanations as features to enhance both the performance and interpretability of LLMs on graphs.

While still representing graphs as textual description, another line of work integrates pretraining strategies with GNNs to enhance the LLMs’ graph reasoning capabilities. For instance, Chen et al. (2023a) proposed two LLM-GNN integration approaches: 1) using LLMs to enhance node features for node classification, and 2) employing LLMs as direct predictors for graph tasks. In line with this, Chandra et al. (2020) utilized LLMs as text encoders and GNNs as graph structure encoders for tasks such as fake news detection in online communities. Similarly, Chen et al. (2023b) explored the possibility of performing node classification without explicit labels by using LLMs to annotate nodes, which are then used in training a GNN model. Likewise, Mavromatis et al. (2023) utilized LLM as a feature extractor for GNNs.

For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graph that combined LLMs and GNNs through a tunable side structure alongside each layer of the LLM, enhancing both training efficiency and inference speed. Similarly, Duan et al. (2023) introduced SimTeG, a simple yet effective approach that enhances textual graph learning by leveraging LLMs. In their method, the LLM is first fine-tuned on a downstream task, such as node classification. Afterward, node embeddings are generated by extracting the last hidden states from the fine-tuned LLM. Additionally, Zhang (2023) proposed Graph-toolformer, which leverages prompt engineering, powered by ChatGPT, to augment LLMs’ graph reasoning capabilities.

While these approaches make LLMs more accessible for graph-related tasks, they fail to fully capture the structural intricacies inherent in graph data. Moreover, their performance hinges on effective prompt engineering, a demanding process that requires careful design and repeated experimentation to obtain optimal results.

Graph Representation as a Learned Projector To address the challenges posed by textual representation approaches, learned projector methods offer an alternative by transforming graph data into embeddings or graph vectors that LLMs can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that retains the general-purpose nature of LLMs while transforming graph structures into graph tokens, a format suitable for LLM processing. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure information into a pre-trained LLM through a tailored disentangled graph neural network layers. Tian et al. (2024) proposed using graph neural prompts to encode knowledge graph, transforming them into graph embeddings that LLMs can leverage during inference. Furthermore, Huang et al. (2023b) introduced a prompt-based node feature extractor (G-Prompt) for few-shot learning on text-attributed graphs. Their method combines a graph adapter (a learnable GNN layer) with task-specific prompts to capture neighborhood information and extract informative node features.

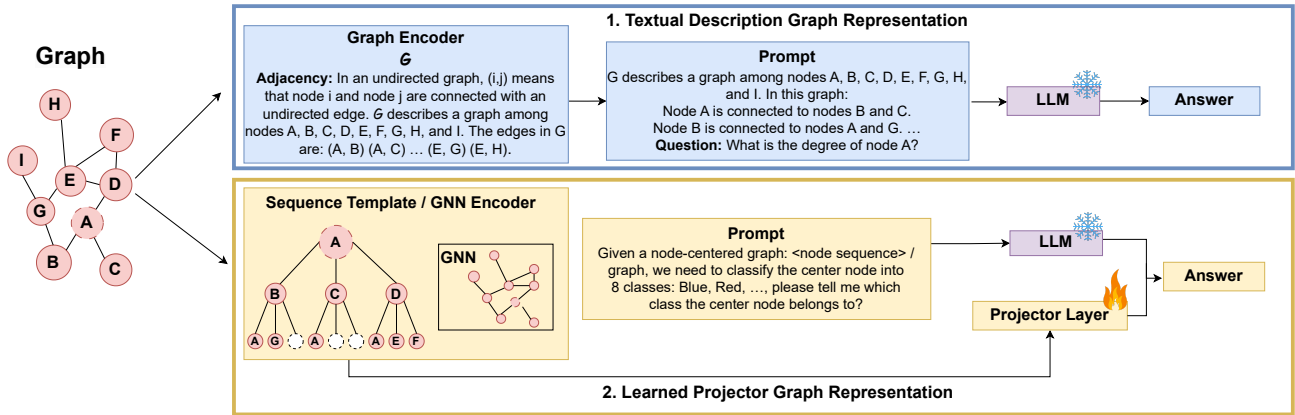


Figure 5: **Graph encoding-based adaptations of LLMs.** In the textual description graph representation, graphs are explained textually (top part). In the learned projector representation, graphs are encoded either by node templates that turns the graph into sequences or by GNNs (bottom part). Note that in both cases, the LLM is frozen. Only the projector that maps the graph into tokens is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs. The learned projector approach captures richer structural information at minimal cost, whereas the textual description approach is limited to simpler graphs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

Another promising line of work condenses graph information into fixed-length prefixes using graph transformers, which are attached to each layer of the LLM to enhance its reasoning capabilities (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers which seamlessly integrates text encoding and graph aggregation in an iterative workflow by nesting GNNs alongside the transformer blocks of a language model. Additionally, Peng et al. (2024) developed ChatGraph, an interactive interface that allows users to interact with their graphs through natural language. Here, graphs are represented as sequential paths and the LLM is fine-tuned on the graph data to support graph analysis.

In this paper, we deliberately exclude projector adaptations that require retraining the LLM, since they inflate computational costs without delivering substantial performance gains. We focus instead on adaptation methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This choice not only slashes computational cost but also affords far greater flexibility in integrating with diverse LLM models, letting us exploit LLMs’ extensive knowledge and capabilities without any retraining overhead while still adapting them to specific graph tasks. Throughout this work, we employ two representative projector methods: **LLAGA** (Chen et al. 2024), which uses node-level templates and a linear projector, and **GRAPH-PROMPTER** (Liu et al. 2024), which employs a GNN-based projector to encode the graph.

This work. In this work, we exclude textual description methods, as they fail to capture the structural intricacies inherent in graph data. Such methods are confined to simpler tasks, like describing basic graph properties such as node degree, and become infeasible for larger graphs: the prompt token limit cannot accommodate the detailed descriptions that large graph structures require, quickly exceeding allowable token counts. From a vulnerability standpoint, textual description methods introduce no unique attack surface. Because the graph is rendered as plain text (e.g., “Node A is connected to Node B”), these methods are exposed only to traditional text-based adversarial attacks, not to the structural or feature perturbations specific to graph-based models. They therefore lack the complexity that would expose them to the novel, graph-specific vulnerabilities at the heart of our investigation into the adversarial robustness of graph-aware LLMs.

A.2 Attacks on GNN

Several types of attacks exist on GNNs, including membership inference attacks (Olatunji, Nejdil, and Khosla 2021; He et al. 2021; Conti et al. 2022), which aim to determine whether a specific node was part of the model’s training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which exploit the GNN’s output and learned embeddings to infer sensitive or missing node attributes; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which seek to extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which attempt to degrade the model’s performance by introducing subtle perturbations to the graph or

node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which aim to reconstruct the underlying graph structure using the learned embeddings or model outputs.

Among these attack types, adversarial attacks are of particular concern because they strike directly at the model’s performance. In adversarial attacks, small, carefully crafted *perturbations*—modifying node features or altering edges—are introduced into the graph to significantly degrade the GNN’s performance. These attacks can be categorized into two types: **poisoning attacks**, which occur during the *training phase* by manipulating the training data, and **evasion attacks**, which occur at *test time*, where the adversary introduces perturbations to the input graph to mislead the model’s predictions. For instance, Zügner, Akbarnejad, and Günnemann (2018) demonstrated that adversarial perturbations on graph structures or node features can significantly degrade the performance of GNNs. Their method, **NETTACK**, employed a greedy optimization technique to compute the minimal perturbations that would maximize the model’s loss function during training. This approach effectively identifies the smallest changes in the features or graph structure needed to disrupt the model’s predictions. Li et al. (2023) revisited the problem of adversarial attacks on graph data from a data distribution perspective, highlighting how adversarial examples can significantly alter the underlying data distribution and proposed a novel defense mechanism to mitigate such effects. Similarly, Wu et al. (2019) provided a comprehensive analysis of adversarial examples on graph data, offering deep insights into both attacks and defenses. They focused on the vulnerabilities of GNNs to small perturbations and proposed defense techniques designed to detect and counteract adversarial modifications, thus enhancing model robustness. **METAATTACK** (Zügner and Günnemann 2019) performed both poison and evasion attacks on GNNs by treating the graph structure as a hyperparameter that can be optimized. The attack directly modifies the graph structure using gradient descent, formulating the problem as a bi-level optimization task. Their method utilized meta-gradients, which capture how small perturbations in the graph structure affect the attacker’s loss after training, enabling precise updates to the graph that degrade the model’s performance.

In this paper, we leverage **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attack methods, encompassing both poisoning and evasion strategies, to conduct our investigation. Specifically, poisoning attacks target model integrity during training, while evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK performs graph structure perturbation by adding and removing edges between nodes, while ensuring that the degree of the perturbed graph is preserved. It uses a greedy optimization technique to compute the minimal perturbations that will maximize the model’s loss function during training, with respect to a surrogate model. This surrogate model is a linearized 2-layer graph convolutional network (GCN), which approximates how changes in the graph structure affect the

classification result. The primary objective of NETTACK is to increase the model’s confidence in incorrect classifications by maximizing the distance between the logits of the perturbed graph and those of the clean graph. To achieve this, NETTACK identifies candidate edges for perturbation around a target node and evaluates a score function, for each connected edge. This score indicates how significantly altering a particular edge will impact the classification result of the target node. The edge with the highest score is then selected, and the graph is updated by either adding or removing that edge. Since NETTACK is a targeted attack (only cause misclassification of a specific single node), we adapt the same method in (Zügner and Günnemann 2019) to convert it into untargeted attack (compromise node classification performance of the model) which randomly select one test node and attack it using NETTACK while considering all nodes in the network. We then aggregate the result over the test nodes.

METAATTACK optimizes the graph structure directly via gradient descent by treating the adjacency matrix (graph structure) as a hyperparameter. It frames the attack as a bi-level optimization problem: the inner level minimizes the training loss on the original graph, while the outer level minimizes the loss on the poisoned graph. To achieve this, METAATTACK utilizes a surrogate model, specifically a linearized two-layer GCN, to approximate how changes in the graph structure affect model performance. A score function evaluates the impact of each perturbation, and the attack flips the sign of meta-gradients (gradient wrt hyperparameters) for connected nodes, indicating which edges to remove. The algorithm then greedily selects perturbations—either adding or removing edge based on those with the highest scores, to maximize the attack’s impact.

For both methods, we use the same unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which ensures that the graph’s degree distribution only changes slightly. Therefore, we only perturb 10% of the edges.

In our poisoning attack experiments with LLAGA, we employ the perturbed graph structure obtained from NETTACK or METAATTACK to generate the node-level sequences and retrain the projector that converts the node-level sequences into a sequence of token embeddings. Similarly, for GRAPH-PROMPTER, we utilize the perturbed graph structure, re-encode the graph using a graph attention network (GAT), and retrain the projector with the perturbed graph. For evasion attacks, the graph-aware LLM is trained on unperturbed data, with perturbations introduced only during the inference stage.

A.4 Algorithm for Node Sequence Template Injection Attacks

We present the algorithm for the node sequence template injection attack in LLAGA in Algorithm 1. The model builds a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14). When a node has fewer than k neighbors, placeholders fill the gap—and that is precisely where the attack surface lies. We exploit it by injecting malicious nodes into the placeholder positions through graph manipulation, using three strate-

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k
Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$ 
       neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

gies: *Non-Adjacent Injection (NI)* inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* injects a single high-degree node into one placeholder

(lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all placeholders (lines 35–43). These uninformative nodes corrupt the node sequence, degrading the projection layer’s performance and impairing its training.

A.5 Ablation Studies

Here, we present the results for the ablation studies presented in the main paper.

Ablation Results: Varying Feature Perturbation In Figure 6, we report the performance of the two graph-aware LLMs, LLAGA and GRAPH-PROMPTER, as the percentage of perturbed features varies. Increasing the proportion of perturbed features consistently and monotonically degrades accuracy for both models, confirming their pronounced sensitivity to feature-level attacks.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 establish that the choice of base LLM has negligible impact on both clean performance and adversarial vulnerability in the LLAGA framework. Across Cora, Citeseer, and PubMed, models built on Vicuna-7B and Llama2-7B achieve nearly identical clean accuracy and degrade almost identically under both poisoning and evasion attacks. Under METAATTACK evasion, for instance, Cora accuracy drops to 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B—a difference well within one standard deviation. The same pattern holds across all datasets and attack types, confirming that attack effectiveness is essentially independent of the base LLM.

A.6 Results on the ArXiv Dataset

We now evaluate our attack on the large-scale ArXiv dataset, which comprises 169,343 nodes and 1,166,243 edges with a sparsity of 0.81. This experiment probes the effectiveness of node sequence template attacks on large, real-world graphs. As Table 9 shows, we find a pronounced gap between poisoning and evasion: poisoning attacks consistently outperform evasion across all three injection strategies, reversing the pattern seen on Cora, Citeseer, and PubMed. This indicates that ArXiv has inherent structural differences that leave it more vulnerable to poisoning yet more robust to evasion. Even at ArXiv’s very low sparsity, inserting a single supernode can markedly affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 illustrate our adversarial attacks on real features from the Cora dataset. The “input” is the text the LLM receives once the attack is applied. Crucially, while a human reader sees the original and perturbed text as visually identical or nearly so, the LLM processes a different sequence of Unicode characters or a different character order, which alters its behavior. We demonstrate this in Table 10 for the homograph attack and Table 11 for the reordering attack.

Feature Corrector Defense Template We designed a feature corrector module to neutralize imperceptible attacks, specifically homograph substitutions and character reorderings. The module is a prompt-based defense built on LLMs, with a prompt carefully engineered to make the LLM act as a linguistic validator that detects and corrects subtle, human-imperceptible perturbations.

The prompt template appears below. By explicitly stating the task and specifying a clear output format, it markedly improves the LLM’s performance.

A.8 Detailed Discussion

Our findings yield several critical insights into the vulnerabilities and design considerations of graph-aware LLMs. As the integration of graph-structured data into LLMs gains momentum, it is essential to examine not only the performance benefits but also the vulnerabilities of these novel architectures. Our systematic investigation of adversarial attacks on two representative graph-aware LLMs, LLAGA and GRAPH-PROMPTER, exposes significant weaknesses, especially under the poisoning and evasion attacks traditionally used to undermine GNNs. We discuss our key observations below.

Impact of Graph Encoding on Adversarial Vulnerability

Our central finding is that the method used to integrate graph structure into LLMs directly increases their susceptibility to adversarial attacks. The node sequence template employed in LLAGA creates a vulnerability that adversaries readily exploit, causing substantial performance degradation. This attack surface, unique to LLAGA, exposes a critical trade-off in graph-aware LLM design: node-level templates are an effective way to incorporate graph structure and boost performance on graph tasks, but they simultaneously open new attack vectors absent from traditional GNNs. GRAPH-PROMPTER, which uses a GNN-based projector for graph encoding, is markedly more resilient to these attacks, underscoring the clear advantages of hybrid architectures that combine the strengths of both LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations

Beyond the vulnerabilities in graph encoding, our results establish the potency of feature perturbation attacks against both graph-aware LLMs. Despite their different graph-encoding schemes, LLAGA and GRAPH-PROMPTER are both acutely sensitive to imperceptible perturbations of node features. This exposes a broader truth about the LLM paradigm: these models excel at handling textual data, yet their robustness to adversarial perturbations—especially those targeting node features—remains a critical weakness. The fact that adversaries can degrade performance by subtly, undetectably altering node attributes fundamentally undermines the integrity of graph-aware LLMs, particularly in security-sensitive applications. Text sanitization techniques such as those in (Boucher et al. 2022) offer partial defenses, but countering imperceptible attacks like homograph-based manipulations remains hard. Our proposed use of LLMs as a corrector directly mitigates these vulnerabilities, prompting the model to detect and repair suspicious or malformed inputs before downstream processing.

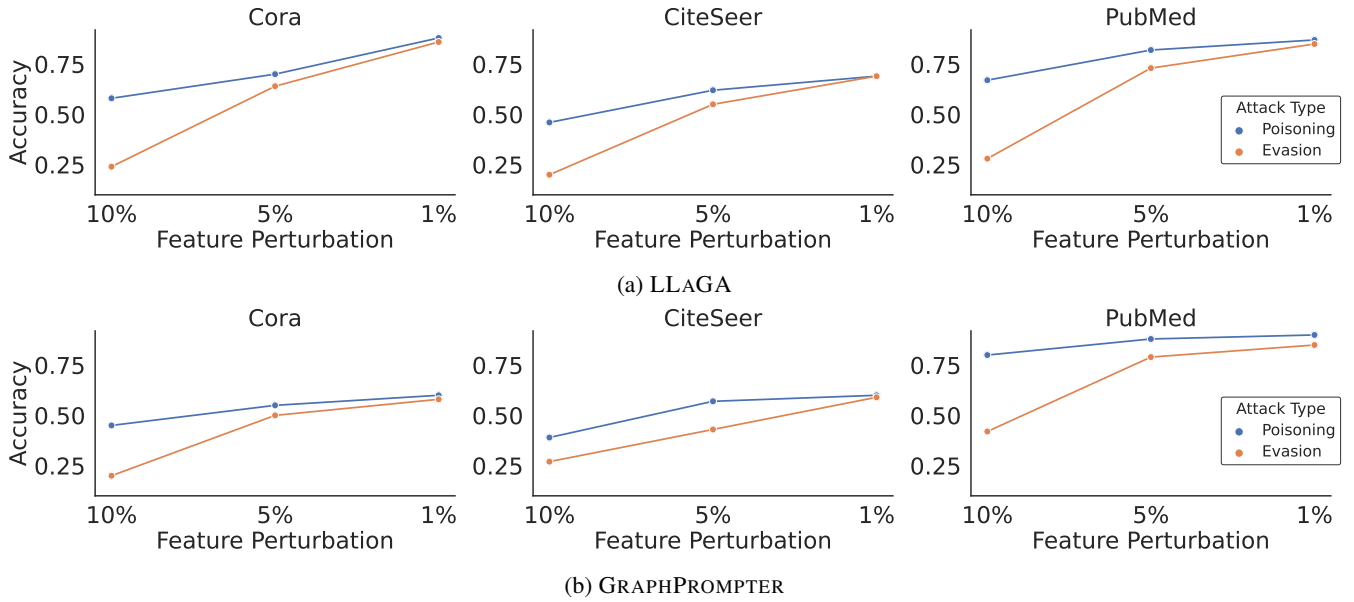


Figure 6: Performance of the Graph-aware LLM when the percentage of the perturbed feature is varied. The lower the accuracy, the better.

Table 8: Impact of base model changes on model performance and on susceptibility under different attacks, shown for LLAGA. The results demonstrate conclusively that the choice of base model does not affect susceptibility to attacks.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our findings establish that the efficacy of poisoning and evasion attacks on graph-aware LLMs varies by dataset. Attacks are highly effective on Cora, Citeseer, and PubMed, whereas ArXiv exhibits a more nuanced interaction between attack strategy and graph structure. This variation makes clear that dataset characteristics are decisive when evaluating the vulnerabilities of graph-aware LLMs. As these models are deployed across diverse real-world settings, their vulnerabilities must be assessed not only in terms of algorithmic design but also with respect to the specific domains and data to which they are applied.

Implications for Graph-aware LLM Development The vulnerabilities exposed in this study carry broad implications for the future development and deployment of graph-aware LLMs. Given their accelerating adoption for tasks once reserved for GNNs, designers must explicitly weigh the trade-offs between performance and vulnerability. As graph-aware LLMs evolve, there is an urgent need for more robust encod-

ing techniques and defenses against adversarial manipulation, particularly against feature perturbations. Moreover, our characterization of how adversarial attacks transfer from GNNs to graph-aware LLMs opens new avenues for protecting these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have concentrated on structural perturbations, because feature-based perturbations typically fail to change the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and are therefore ineffective. As a direct result, there are *no known defenses against feature-based attacks on GNNs*. Yet, as we demonstrate in the main paper, feature perturbation attacks are decisively effective in the graph-aware LLM paradigm. This creates a genuinely new challenge: feature-based attacks that were harmless against GNNs now successfully manipulate model behavior, making defenses that address

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgpprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llction plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: The prompt template used for the LLM-based feature corrector module. It explicitly defines the role, task, and expected output format for the LLM.

Table 9: Results of the node sequence template injection attack across datasets and methods. *NI*, *SI*, and *MSI* denote the non-adjacent, single supernode, and multiple supernode injection attacks, respectively. Clean refers to model performance without any attack. On ArXiv, poisoning outpaces evasion, reversing the pattern seen on the smaller graphs.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 ± 0.07	————
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	————	0.71 ± 0.04	————
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	————	0.90 ± 0.04	————
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	————	0.73 ± 0.02	————
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

both structural and feature-based perturbations essential. Text sanitization techniques—such as optical character recognition (OCR) as a preprocessing step before encoding text for the LLM—offer only partial protection, since OCR is imperfect and misinterprets homoglyphs at a higher rate than unperturbed text (Boucher et al. 2022). This limitation motivated our design of an LLM-based self-correcting mechanism for feature perturbation.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks transfer from GNNs to graph-aware LLMs, particularly in evasion attacks, one might expect GNN defenses to carry over as well. This expectation is wrong. Unlike traditional GNN settings, *graph-aware LLMs are also susceptible to feature perturbations, which renders these defenses ineffective.*

One prominent GNN defense makes the adjacency matrix trainable, letting edge weights adapt during training to absorb adversarial perturbations by assigning lower weights to edges between dissimilar nodes and higher weights to edges between nodes with greater feature similarity. Several **preprocessing-based** defenses build on this idea, removing edges between nodes with low feature similarity (Wu et al. 2019) or introducing randomized edge dropping (Dai et al. 2018). All of them depend critically on clean, unperturbed node features to compute reliable similarity scores. When node features are themselves perturbed, as in our attacks, these defenses not only fail but can actively worsen the problem by wiring up incorrect edges. We make this failure concrete on a random node in the Cora dataset (Figure 8): the clean feature similarity graph preserves some original edges while introducing a few wrong connections, but the edges reconstructed from perturbed features preserve none of the original connections, yielding entirely incorrect reconstructions.

Another line of work decouples reliance on the graph structure. (Shanthamallu, Thiagarajan, and Spanias 2021) trains a surrogate predictor that depends solely on node features and uses an uncertainty matching strategy to extract graph information from the GNN, while Wu et al. (2021) introduced a co-training framework that trains submodels independently on feature and structural views, enabling knowledge distillation by adding confident unlabeled data to the training set. Both approaches, however, again assume access to unperturbed node features—an assumption that fails outright in our setting, where structure and features are perturbed together. Consequently, structural defense strategies developed for GNNs simply do not transfer to the graph-aware LLM paradigm, which demands novel, purpose-built defenses.

Table 10: An illustration of the imperceptible homoglyph attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtln squnc pttrns Abstract: Svrl cmputr lgrithms fr discvring pttrns in grups f prtln squncs r in us tht r bsd on fitting th prmtrs f stttisticl mdl to grup of rltd sauncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: An illustration of the imperceptible reordering attack. While the perturbed text is processed as distinct characters by the LLM, the visual output is nearly identical for a human, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplclpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

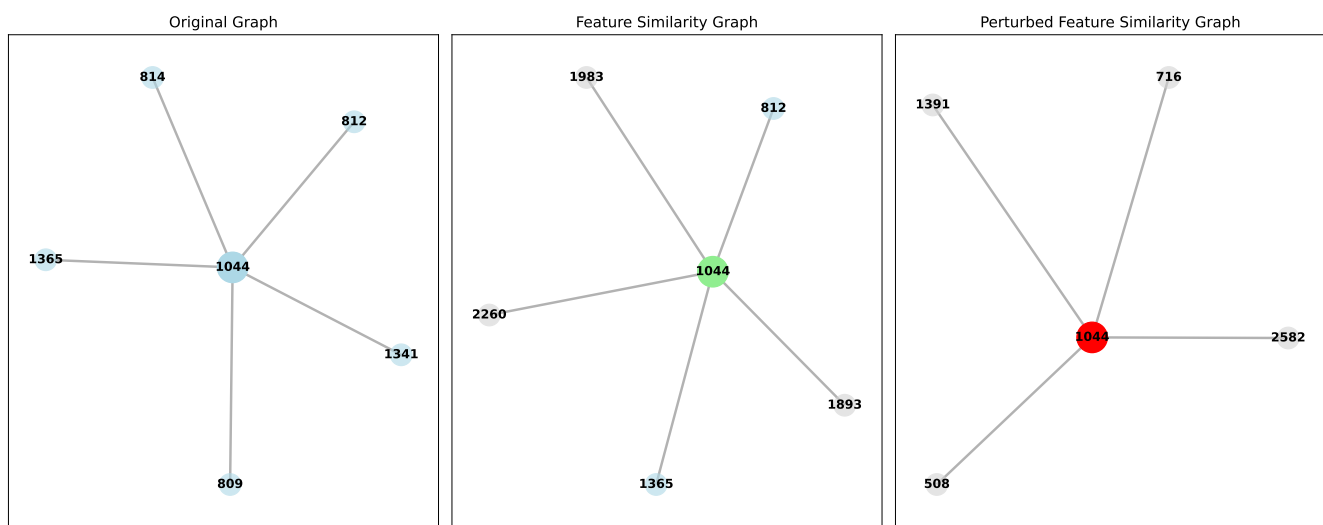


Figure 8: Edges reconstructed for a random node 1044. **Original (left)** is the ground truth edges from the original graph. **Feature Similarity (middle)** shows the edges reconstructed using clean node features, where blue nodes indicate preserved connections from the original graph. **Perturbed Feature Similarity (right)** shows the edges reconstructed using perturbed node features.